



تمرینات numpy - هفته سوم

تمرین اول- داده‌های مشتریان باشگاه به صورت زیر هست:

```
gym_data = np.array([
    [28, 75, 175, 4],
    [34, 68, 168, 3],
    [45, 82, 180, 2],
    [22, 58, 162, 5],
    [38, 90, 0, 1],
    [29, 65, 170, 0]
])

# ستون‌ها به ترتیب:
# ستون 0: سن (سال)
# ستون 1: وزن (کیلوگرم)
# ستون 2: قد (سانتی‌متر)
# ستون 3: تعداد جلسات در هفته

member_names = np.array(["Ali", "Sara", "Reza", "Neda", "Hassan",
                           "Maryam"])
```

۱. در ستون‌های وزن، قد و تعداد جلسات، عدد صفر به معنی اطلاعات گم‌شده است. این مقادیر را با یک عدد مناسب و حسی که از بقیه داده‌ها میگیرید، جایگزین کنید. ستون سن را دست نزنید.

۲. از سه ستون وزن، قد و تعداد جلسات، یک آرایه‌ی جدید بسازید.

۳. برای هر عضو باشگاه، یک عدد به عنوان «نمره‌ی آمادگی جسمانی» محاسبه کنید که ترکیبی از شاخص توده بدنی (BMI) و تعداد جلسات در هفته باشد. این نمره را به آرایه‌ی جدید اضافه کنید.
$$\text{نمره} = (\text{BMI} \times \text{ضریب}_1) + (\text{تعداد جلسات} \times \text{ضریب}_2)$$

۴. مشخص کنید کدام عضو بالاترین نمره‌ی آمادگی را دارد و اسمش را چاپ کنید.

۵. مشخص کنید کدام عضو از نظر تعداد جلسات در هفته، بیشترین تفاوت را با بقیه دارد. برای تشخیص این تفاوت، از یک معیار انحراف معیار استفاده کنید.

تمرین دوم - پیشنهاددهنده غذا برای چند نفر همزمان

این آرایه رو در نظر بگیرید:

```
recipes = np.array([
    [15, 350, 2, 5],
    [45, 600, 7, 10],
    [10, 200, 0, 3],
    [30, 450, 5, 7],
    [60, 800, 8, 12]
])
recipe_names = ["Salad", "Curry", "Toast", "Pasta", "Stew"]
```

ستون ها به ترتیب زمان پخت (دقیقه)، کالری، تندى (۱ تا ۱۰)، تعداد مواد اولیه هستند.

۱. اول داده ها رو با Min-Max Scaling به بازه صفر تا یک ببرید تا همه ویژگی ها قابل مقایسه بشن. فرمول $X - \min$ (تقسیم بر $\max - \min$) هست و $\min$ و $\max$ هر ستون رو با $\text{axis}=0$ حساب کنید.

۲. سه مشتری با سلیقه های متفاوت داریم:

```
users = np.array([
    [10, 250, 1, 4],
    [50, 700, 8, 11],
    [25, 400, 4, 6]
])
```

این کاربران رو با همون $\min$ و $\max$ که از مرحله قبل ذخیره کردید، مقیاسسازی کنید.

۳. حالا فاصله اقلیدسی هر کاربر رو با هر غذا حساب کنید. نتیجه باید یک ماتریس $(5, 3)$ باشه که سطرها کاربران و ستون ها غذاها هستن. برای این کار هیچ حلقه `for` ای ننویسید. (`reshape` و `broadcasting`)

۴. برای هر کاربر، نام نزدیکترین غذا به سلیقه ش رو با `np.argmin` پیدا کنید و چاپ کنید.

۵. برای هر کاربر، همه غذاها رو از نزدیکترین تا دورترین رتبه بندی کنید و چاپ کنید.

تمرین سوم - دبیرستان و سناریوهای نمره دهی

این آرایه رو داریم که نمرات چهار دانش آموز هست:

```
scores = np.array([
    [18, 15, 20],
    [12, 14, 16],
    [20, 19, 18],
    [10, 8, 15]
])
```

هر سطر یک دانش آموزه و ستون ها به ترتیب کوییز، میانترم و فاینال هستن.
سه روش وزن دهی:

```
scheme_A = np.array([0.5, 0.3, 0.2])
scheme_B = np.array([0.2, 0.3, 0.5])
scheme_C = np.array([0.1, 0.2, 0.7])
```

۱. سه روش وزن دهی رو توی یک ماتریس به اسم `weight_matrix` با شکل (۳,۳) کنار هم بچینید. سطر اول A، دوم B و سوم C باشه.

۲. نمره نهایی هر دانش آموز رو برای هر سه روش با یک ضرب ماتریسی حساب کنید:

```
final_scores = scores @ weight_matrix.T
```

شکل خروجی باید (۴,۳) باشه. سطرها دانشآموزان و ستونها روشهای A و B و C هستن.

۳. برای هر دانشآموز مشخص کنید کدوم روش بیشترین نمره رو بهش داده. از `np.argmax` با `axis=1` استفاده کنید. خروجی یک آرایه ۴ تایی هست که هر عدد نشاندهنده روش برتر هست (۰ یعنی A، ۱ یعنی B، ۲ یعنی C).

۴. مدیر میخواد یک روش واحد برای کل کلاس انتخاب کنه که میانگین کل کلاس رو بیشینه کنه. از ماتریس `final_scores` استفاده کنید. اول با `axis=0` میانگین هر ستون (یعنی هر روش) رو حساب کنید. بعد با `np.argmax` روشی که بیشترین میانگین رو داره پیدا کنید.

۵. یک روش چهارم هم اضافه شده که وزن هایش هنوز مشخص نیست. فقط میدونیم که مجموع سه وزن باید بشه یک و وزن فاینال دقیقاً دو برابر وزن کوییز هست. یک بردار وزن معتبر برای این روش بسازید و اون رو هم به جدول مرحله ۱ اضافه کنید و چاپش کنید.

تمرین چهارم - یک باگ در خط لوله داده های هواشناسی

این کد رو یک همکار نوشته:

```
weather_data = np.random.randn(2, 8, 4) * 5 + 20
```

این یعنی دو روز، هر روز هشت اندازه گیری، هر اندازه گیری از چهار سنسور.

یک تابع به اسم `analyze_day` داریم که ورودی $(4, 8)$ میخواد یعنی سطرها: سنسورها و ستون ها: زمان. این تابع رو قرار نیست تغییر بدید چون بقیه تیم هم ازش استفاده میکنن.
۱.

```
day2 = weather_data[1]
```

رو استخراج کنید. شکلش $(8, 4)$ هست. اون رو به شکل $(4, 8)$ تبدیل کنید و به تابع `analyze_day` بدید. تابع رو صدا بزنید تا اجرا بشه.

۲. حالا کل `weather_data` رو به یک بردار یک بعدی از ۶۴ عدد تبدیل کنید تا توی فایل ذخیره بشه. از

```
reshape(-1)
```

یا `flatten` استفاده کنید. شکل نهایی رو چاپ کنید. باید

```
(64,)
```

باشه.

۳. یک همکار میگه ترانواده کردن و فلات کردن هر دو داده رو مرتب میکنن و فرقی ندارن. با یک مثال ساده مثلاً یک ماتریس ۲ در ۲ توضیح بدید که چرا این حرف غلطه. اگر از کد استفاده میکنید، خروجی هر دو رو نشون بدید و توضیحی برایش بنویسید.

۴. فرض کنید یک روز سوم هم به داده ها اضافه شده:

```
day3 = np.random.randn(8, 4).
```

کاری کنید که `weather_data` سه روزه بشه با شکل $(3, 8, 4)$ بدون اینکه از اول آرایه رو دستی بازنویسی کنید.

تمرین پنجم - دستیار یادداشت‌برداری هوشمند

۱.

```
importance = np.array([3, 8, 1, 9, 4, 7])
```

یک تابع دیگه توی پروژه هست که ورودی ستونی با شکل (۶,۱) میخواد. importance رو به شکل (۶,۱) تبدیل کنید. این کار رو به دو روش مختلف انجام بدید و شکل نهایی هر دو رو چاپ کنید.

۲.

```
model_out = np.array([[[0.9]]])
```

این آرایه شکل (۱,۱,۱) داره. میخواید مقدار ۰.۹ رو به عنوان یک عدد ساده (نه آرایه) توی یک پیام نشون بدید. مقدار رو به دو روش استخراج کنید: یکی با اندیس گذاری و یکی با متد item(). هر دو رو امتحان کنید. ۳. این کد رو بدون اجرا کردن بخونید و حدس بزنید خروجی

```
print(notes)
```

چی میشه:

```
notes = np.array([5, 10, 15, 20, 25])
pinned = notes[1:3]
pinned[0] = 999
print(notes)
```

پیش بینی خودتون رو بنویسید، بعد کد رو اجرا کنید. اگر نتیجه با پیش بینیتون تفاوت داشت، یک نسخه دوم از کد بنویسید که توش تغییر pinned روی notes اصلی اثری نذاره. برای این کار از copy استفاده کنید.

۴. حالا برعکسش رو امتحان کنید. کدی بنویسید که توش عمداً میخواید تغییر یک زیرمجموعه از داده روی آرایه اصلی اثر بذاره. یعنی اینبار رفتار پیشفرض NumPy دقیقاً همون چیزیه که شما میخواید.

تمرین ششم - مسابقه دوی ماراتن

این مسابقه رو در نظر بگیرید:

```
bib_numbers = np.array([101, 102, 103, 104, 105, 106])
times_5k = np.array([22.3, 25.1, 21.8, 26.4, 23.0, 24.7])
```

۱. یک گروه جدید از دوندگاران را ثبت نام کنید:

```
bib_numbers_2 = np.array([107, 108])
times_5k_2 = np.array([20.5, 27.9])
```

این دوندگاران جدید رو به آرایه‌های اصلی اضافه کنید طوری که هر شماره بیب با زمان خودش جفت بمونه.

۲. حالا برای اعلام نتایج، ترتیب دوندگاران رو کاملاً تصادفی به هم بریزید ولی باز هم بیب هر کس با زمان خودش جفت باشه.

۳. رتبه‌نهایی هر دوندگاران رو از سریع‌ترین تا کندترین پیدا کنید و یک جدول رتبه‌بندی چاپ کنید که شماره بیب و زمان هر نفر رو به همراه رتبه‌شان نشون بده.

۴. یک دوندگاران با بیب ۱۰۴ اعتراض کرده که رتبه‌شان اشتباه ثبت شده. کدی بنویسید که رتبه دقیق این بیب رو بدون اینکه دستی به جدول نگاه کنید، پیدا کنه و چاپ کنه.

تمرین هفتم - یک تصمیم ساده با یک لایه کوچک

یک اپ ساده میخواد بر اساس سه ویژگی یک پیام تشخیص بده که پیام آرامه یا پرانرژی. ویژگی ها عبارتند از طول پیام، تعداد علامت تعجب و تعداد کلمات بزرگ(به ترتیب ستون ها از چپ به راست).

```
X_messages = np.array([
    [12, 0, 1],
    [45, 5, 8],
    [8, 0, 0],
    [30, 3, 4]
])
w = np.array([0.1, 0.8, 0.5])
b = -2.0
```

۱. خروجی خام این نورون ساده رو برای هر چهار پیام حساب کنید.

۲. طبق قانون این نوع نورون ها، خروجی نباید هیچوقت منفی بشه. قبل از گزارش نتیجه نهایی، این مشکل رو حل کنید.

۳. بر اساس نتیجه نهایی، برای هر پیام بگید که آرامه یا پرانرژی. خط جداسازی رو خودتون با یک آستانه منطقی تعریف کنید.